

Observability and Production Engineering

Contents

Foundations

1. [Monitoring and observability](#)
2. [Instrumenting a service](#)
3. [The OpenTelemetry pipeline](#)

Metrics

4. [Metric types](#)
5. [Cardinality](#)
6. [RED and USE](#)

Logs

7. [Structured logging](#)
8. [Volume, levels and cost](#)

Traces

9. [Tracing in practice](#)

Reading the numbers

10. [Percentiles and the tail](#)
11. [Coordinated omission](#)

Diagnosing a Java service

12. [What to capture before restarting](#)
13. [Thread dumps](#)
14. [Heap dumps and memory leaks](#)
15. [Garbage collection](#)
16. [Profiling and flame graphs](#)

Proving it works

17. [Load testing](#)
18. [Chaos engineering](#)
19. [Production readiness](#)

Reference

- 20. [Interview questions](#)
- 21. [Common mistakes](#)
- 22. [Quick reference](#)

1. Monitoring and observability

Monitoring is watching signals chosen in advance for failure modes anticipated in advance. Someone decided what to measure and what "bad" looks like, and the result is dashboards and alerts.

Observability is the property of being able to work out what is happening inside a system from its external outputs, **including for questions nobody thought to ask beforehand**. The term is borrowed from control theory, where a system is observable if its internal state can be determined from its outputs.

The distinction that survives questioning: monitoring answers **is it broken**, observability answers **why**. Monitoring handles known unknowns, since we know a database can get slow so we watch it. Observability handles unknown unknowns, where a combination of conditions nobody predicted produces behaviour nobody has a dashboard for.

The practical test is a good one to offer: **can a new question be answered without shipping new code?** If every investigation begins with adding a metric and waiting for a deploy, the system is monitored rather than observable.

Treat that contrast as interview shorthand rather than a boundary. Monitoring can point at a cause when the cause is one somebody anticipated, and observability data is often what tells us something is wrong in the first place. The honest form is that monitoring reports whether a known failure condition is occurring, and observability supplies the data and context to investigate whatever we find.

Worth being honest about: observability is also a marketing word now, and collecting three kinds of telemetry does not confer it. What confers it is having enough detail, with enough context attached, to slice the data along a dimension nobody anticipated.

2. Instrumenting a service

What to instrument, roughly in descending order of value:

1. **Every inbound request**: rate, errors, duration, broken down by endpoint and status.
2. **Every outbound dependency call**: the same four, per dependency. This is what tells us whose fault a slowdown is.
3. **Queue depth and consumer lag**, wherever work is asynchronous.
4. **Saturation of bounded resources**: connection pool usage, thread pool queue depth, in-flight requests.
5. **Business events that matter**: orders placed, payments declined, signups completed.

The principle underneath: **instrument the boundaries first**. Most diagnostic value comes from knowing exactly what crossed into and out of the service, because that is what separates "our problem" from "their problem".

Separate from the measurements themselves, every signal should carry **resource attributes** identifying where it came from: service name, version, instance or pod, environment and region. These are set once for the process rather than per measurement, and they are what make "only the pods running version 4.7 are slow" a question that can be answered at all. They are not an exception to the cardinality rule in section 5; they are a small fixed set, and the version one changes only at deployment.

In Java the usual arrangement is **Micrometer** as a vendor-neutral facade for metrics, in the way SLF4J is for logging, with **OpenTelemetry** for traces and Micrometer Tracing bridging the two. The exact wiring shifts between Spring Boot versions, so it is worth knowing the roles rather than memorising the configuration.

Most of it needs no code. The OpenTelemetry Java agent instruments many common libraries automatically, so HTTP servers and clients, JDBC, Kafka and others produce telemetry, typically spans and, where the instrumentation supports it, metrics, without a line being written. Manual instrumentation is for what the agent cannot know, which is domain-specific:

```
@Service
public class OrderService {

    private final Timer createTimer;
    private final Counter rejected;

    OrderService(MeterRegistry registry) {
        this.createTimer = Timer.builder("orders.create")
            .description("Time to create an order")
            .publishPercentileHistogram() // export buckets, see section 4
            .register(registry);

        // The label value is bounded: a small fixed set of reasons.
        this.rejected = registry.counter("orders.rejected", "reason",
            "insufficient_funds");
    }

    public Order create(CreateOrderCommand command) {
        return createTimer.record(() -> doCreate(command));
    }
}
```

Naming is worth settling early, because renaming a metric breaks every dashboard and alert built on it. Pick a consistent hierarchy, keep the unit in the name or in metadata, and never encode a variable into the metric name where it belongs as a label.

3. The OpenTelemetry pipeline

Worth knowing as a shape, because interviewers ask where telemetry goes after the application produces it, and the answer is the same regardless of which backend a company bought.

```
Application
  | instrumentation: the agent, the SDK, or our own code
  v
OpenTelemetry SDK      batch, resource attributes, head sampling
  | OTLP, the OpenTelemetry Protocol, over gRPC or HTTP
  v
Collector              filter, redact, tail sample
  | exporter, per backend
  v
Backend (one or several, and they can differ per signal)
```

The five parts and what each is for:

Part	Role
Instrumentation	Generates telemetry from the application: spans, metrics and, where it is configured, logs. The Java agent covers common libraries; the SDK is for our own code
SDK	Creates and batches telemetry, attaches resource attributes, can apply head-based sampling, and hands off to an exporter
OTLP, the OpenTelemetry Protocol	The protocol used to carry telemetry between these components. It transports over gRPC or HTTP, and covers all three signals, which is what makes the rest interchangeable
Collector	A separate process that receives, processes and forwards: filtering, redacting, tail sampling where configured, and exporting. Optional
Exporter	Sends to a specific backend. Swapping backends is an exporter change

Note where the two kinds of sampling live, because it is a common follow-up: **head-based sampling can happen in the SDK, and tail-based sampling belongs in the collector**, since only the collector sees the whole trace before deciding.

The collector is the component worth arguing for in a larger deployment. Without it, every service holds backend credentials, knows the backend's address, and has to be redeployed to change either. With it, the application only knows how to speak OTLP to a local endpoint, and everything else, which includes redacting fields, dropping noisy metrics, tail sampling, and sending one signal to two backends during a migration, is configuration in one place.

It usually runs either as a sidecar next to the application or as a shared deployment per cluster.

The point of the standard is that instrumentation is no longer vendor-specific. Changing observability vendor used to mean re-instrumenting every service; with OTLP in the middle it is a collector configuration change.

4. Metric types

Type	Behaviour	Suits
Counter	Only increases, resets on restart	Requests, errors, bytes sent
Gauge	Goes up and down, sampled at a point in time	Queue depth, threads in use, memory
Histogram	Counts observations into buckets	Latency, payload size
Summary	Computes quantiles inside the process	Latency, with an important caveat

Counters are queried as rates, not as values. The raw number resets when the process restarts, so a dashboard showing the counter itself shows a sawtooth that means nothing. The rate of change is the useful quantity.

Histogram against summary, and why it matters

A **summary** calculates quantiles inside each process. The p99 it reports is exact for that instance, and **quantiles from separate instances cannot be combined**. Averaging the p99 from ten pods produces a number with no statistical meaning, and it is usually lower than the true p99 across all traffic.

A **histogram** exports counts per bucket. Buckets from every instance add together cleanly, so the percentile is estimated at query time from the combined distribution. That estimate is an approximation, and its accuracy depends on where the bucket boundaries fall, but it describes the whole service rather than one instance.

In Micrometer that is the difference between `publishPercentileHistogram()`, which exports buckets, and `percentiles(0.99)`, which computes client-side. For service-level latency across several instances, histograms are usually the better choice.

The cost of a histogram is a time series per bucket, which is where the next section comes in.

5. Cardinality

Cardinality is the number of distinct time series, and it is the product of every combination of label values. It is the most common way a metrics system is destroyed by accident.

```
http_requests_total{method, status, endpoint}
  7 methods × 20 statuses × 50 endpoints = 7,000 series      fine

http_requests_total{method, status, endpoint, user_id}
  7 × 20 × 50 × 1,000,000 users          = 7 billion series  catastrophic
```

Each series carries its own storage, its own memory in the backend, and its own cost. A single unbounded label turns a working system into an outage of the monitoring itself, usually at the moment it is most needed.

Never use as a label: user identifier, request identifier, session identifier, email address, trace identifier, a full URL with query parameters, an error message, or a timestamp.

Safe labels are bounded and small: HTTP method, status code or status class, **endpoint template** rather than the resolved path, region, service name, and a small enumerated reason.

The endpoint one is the trap in practice. `/orders/12345` as a label value creates one series per order. `/orders/{id}` creates one series. Frameworks usually expose the template, and using it is the difference between a working dashboard and an unusable one.

High-cardinality investigative detail generally belongs in logs and traces, which are built to hold it, subject to their own cost controls and to the same rules about personal data. That is the real division of labour between the signals: metrics answer aggregate questions cheaply at low cardinality, and logs and traces answer specific questions expensively at high cardinality.

6. RED and USE

Two complementary methods, and knowing which applies to what is the point.

RED, from Tom Wilkie, for **request-driven services**:

	Meaning
Rate	Requests per second
Errors	Failed requests per second, and as a proportion
Duration	The latency distribution, as percentiles

USE, from Brendan Gregg, for **resources**, meaning anything with a finite capacity:

	Meaning
Utilisation	The proportion of time the resource was busy
Saturation	The work that had to wait, such as queue length
Errors	Error events for that resource

RED describes what users experience; USE describes what is underneath and explains why. A service can be at 100% utilisation on a connection pool while its RED metrics still look acceptable, and saturation is the signal that gives warning before the service degrades.

Google's **four golden signals** are latency, traffic, errors and saturation, which is RED plus the saturation term from USE.

7. Structured logging

A log line is **data, not prose**. Structured logging means emitting key and value pairs, usually as JSON, so a query engine can filter and aggregate without regular expressions.

```
// Unstructured. Finding slow orders means writing a regex against English.
log.info("Order " + id + " created for customer " + customerId + " in " + ms + "ms");

// Structured. Queryable by any field.
log.atInfo()
    .addKeyValue("orderId", id)
    .addKeyValue("customerId", customerId)
    .addKeyValue("durationMs", ms)
    .log("order created");
```

The message stays a short constant, which means it can be grouped. Everything variable becomes a field.

Context that applies to every line in a request belongs in the MDC, the mapped diagnostic context, so it is attached automatically rather than passed into every call:

```
try (var ignored = MDC.putCloseable("traceId", traceId);
     var ignored2 = MDC.putCloseable("tenantId", tenantId)) {
    orders.create(command);
}
```

That is what makes logs joinable to traces, and it is the single highest-value line of logging configuration in a distributed system.

In a current stack the trace identifier should not be one of the fields we set by hand. Micrometer Tracing and the OpenTelemetry agent put the active trace and span identifiers into the MDC themselves, so the logging pattern only has to reference them. Reserve manual MDC entries for request-scoped fields the framework has no way to know, such as the tenant or the customer, and let the correlation identifiers arrive on their own.

What belongs in a log line: the identifiers needed to correlate it, the decision that was made, and the inputs that determined the decision. What does not: credentials or tokens, personal data beyond what is necessary, and anything a metric already answers.

8. Volume, levels and cost

Log levels only work if they mean something consistent:

Level	Should mean
ERROR	Something failed and a human needs to look
WARN	Something unexpected happened and the system handled it

Level	Should mean
INFO	A significant event, sparse enough to read
DEBUG	Detail for diagnosis, usually off in production
TRACE	Very fine detail, rarely on anywhere

The discipline that keeps them useful: **if every request emits twenty INFO lines, INFO has become DEBUG** and nobody can read the log any more. INFO should mark meaningful operational events, and stay sparse enough to still be readable during an investigation.

Logs easily become the most expensive signal, because volume grows directly with traffic and retention is often measured in weeks. Traces can overtake them where sampling is generous and retention is long, so this is a matter of configuration rather than a fixed ranking. Three ways to control it:

- **Sample.** Keep every error, and a fraction of successful requests. The errors are what get read.
- **Tier retention.** Recent logs searchable and hot, older ones in cheap storage, oldest deleted.
- **Do not log what a metric already counts.** A line per request, kept for weeks, in order to count requests, costs far more than a counter that does the same job.

The division worth memorising: **metrics for what, logs for why, traces for where.**

9. Tracing in practice

The distributed systems note covers traces, spans and propagation. What follows is what it takes to make tracing actually work, which is mostly about the places it silently breaks.

Context propagation is the whole mechanism. A trace exists only because every service passes the identifier onward, usually in the `traceparent` header from the W3C Trace Context standard. One service that drops it ends the trace there, and the effect is a trace that mysteriously stops halfway with no error anywhere.

Asynchronous boundaries are where it breaks, and this is the Java-specific part. In synchronous code the current trace context is associated with the executing thread, commonly through thread-local storage, so any handoff to another thread has to carry the context across explicitly:

- Submitting work to an `ExecutorService` loses the context unless the executor is wrapped.
- A `CompletableFuture` continuing on another thread loses it the same way.
- Publishing to a queue loses it unless the context is written into the message headers and read back by the consumer.

The OpenTelemetry agent wraps most standard executors, and anything hand-rolled needs doing explicitly. When a trace stops at a service that clearly did the work, an async handoff is the first place to look.

Sampling decides what fraction is kept, because recording everything is expensive at volume. **Head-based** sampling decides at the start of the request, which is cheap and keeps a random selection. **Tail-based** sampling decides once the whole trace is collected, so it can keep the slow and failed ones, at the cost of buffering everything until the decision is made. Tail-based sampling keeps the rare slow and failed traces that head-based sampling discards, and pays for that with collector memory, buffering and a more complex system to operate. Head-based sampling is adequate for a great many services, so this is a trade rather than a ranking.

Unlike metrics, **spans can carry high-cardinality attributes**, and should: user identifier, order identifier, the query that ran. That is the point of the signal. The same limits apply as for logs, though, so personal data and query contents need redacting or handling under whatever privacy rules cover the service. "It is a trace, not a log" is not an exemption, and an interviewer who hears the attribute list will often ask about it next.

10. Percentiles and the tail

An average hides everything that matters. A service with a mean latency of 10 milliseconds can have a p99 of two seconds, and the mean will not move when the tail doubles.

Two reasons the tail matters more than it first appears.

A page is many requests. If rendering one page makes 20 calls and each has a 1% chance of exceeding the p99 threshold, then assuming those events are independent, the chance that at least one does is $1 - 0.99^{20} \approx 18\%$. The p99 of a single request is close to the p82 of a page, so what looks like a rare event for the service is a routine event for the user.

Percentiles must never be averaged. The mean of the p99 from ten instances is not the p99 of the service. That is the histogram-against-summary point from section 4, and it is the most common way a dashboard reports a comfortable number while users experience something else.

Which percentile for what:

Percentile	Use
p50, the median	Typical experience. Useful, and not what to alert on
p90 or p95	The beginning of the tail, where problems appear first
p99	A common latency objective for user-facing services
p99.9	Useful where very low tail latency matters, particularly in systems with high fan-out
Maximum	One sample, and usually noise

11. Coordinated omission

The measurement error that makes load test results look far better than reality. The name is Gil Tene's, and it is worth being able to explain because it invalidates a great deal of published benchmarking.

A typical load generator sends a request, waits for the response, then sends the next one. When the system stalls for a second, the generator stalls too, so it **sends fewer requests during exactly the period when latency was worst.**

Intended: send a request every 10 ms

Reality: request at $t=0$ takes 1000 ms
the generator waits, so nothing is sent from $t=0$ to $t=1000$
100 requests that should have been sent were never sent
and none of their 1000-to-0 ms waits were recorded

Recorded distribution: one slow sample

Actual user experience: 100 requests queued behind a one-second stall

The recorded distribution omits the bad part, and the omission is coordinated with the badness, which is where the name comes from.

The correction is to measure latency **against the intended send time** rather than the actual one, so a request that could not be sent still accrues the waiting it should have. Load generators that handle this correctly include wrk2, and HdrHistogram provides the correction directly.

The related effect on the service side is **tail latency amplification**. A request that fans out to 10 services, each with a p99 of one second, has a $1 - 0.99^{10} \approx 9.6\%$ chance that at least one of them is slow, assuming the slow events are independent. Read that as an approximation rather than a conversion rule: under independence, what is a one-in-a-hundred event for a single dependency becomes close to a one-in-ten event for the combined request, so the tail we were treating as rare is not rare at the level the user experiences. Real dependencies are often correlated, sharing a database or a network path, which usually makes the true figure worse rather than better. That is the argument for hedged requests, and for caring about p99.9 in a system that fans out.

12. What to capture before restarting

The single most practical thing in this document. **A restart destroys the in-process evidence**, and the pressure during an incident is always to restart first. Metrics, logs and traces already sent survive, so the restart does not leave us blind; what it takes away is everything that only exists inside the running process, which is exactly what the following sections read.

For a sick JVM, capture in this order:

```
jcmd <pid> Thread.print > threads-1.txt      # repeat 3 times, 5 seconds apart
jcmd <pid> GC.heap_info                        # occupancy by generation
jcmd <pid> VM.uptime
jcmd <pid> JFR.start duration=60s filename=/tmp/recording.jfr

# Only if memory is the suspected problem: this pauses the JVM and the
# file is roughly the size of the live heap.
jcmd <pid> GC.heap_dump /tmp/heap.hprof
```

Three thread dumps rather than one, because a single dump cannot distinguish a thread that is stuck from a thread that is merely busy at that instant.

Two configuration items that should already be set, because they capture evidence without anyone being present:

```
-XX:+HeapDumpOnOutOfMemoryError
-XX:HeapDumpPath=/dumps
-Xlog:gc*:file=/logs/gc.log:time,uptime:filecount=5,filesize=20M
```

In Kubernetes this gets harder, because a failing liveness probe restarts the container and everything in its filesystem goes with it. Capturing evidence needs a volume that survives the restart, a `preStop` hook that dumps before termination, or a termination grace period long enough to collect anything at all.

13. Thread dumps

A thread dump lists every thread with its stack and state. The states worth recognising:

State	Meaning
<code>RUNNABLE</code>	Executing, or inside a native operation such as socket input and output. It does not by itself mean the thread is using the processor
<code>BLOCKED</code>	Waiting to acquire a monitor another thread holds
<code>WAITING</code>	Waiting indefinitely, such as on <code>Object.wait</code> or a queue take
<code>TIMED_WAITING</code>	The same with a timeout, including <code>sleep</code>

Three patterns cover most diagnoses:

Many threads `BLOCKED` on the same lock. Contention. The dump names the lock and which thread holds it, and that thread's stack is where the fix belongs.

Many threads `RUNNABLE` inside `socketRead`. Almost certainly blocked in native input and output, waiting on a remote call rather than using the processor. This is the thread-pool exhaustion pattern from the API note, and it is what a missing timeout looks like from the inside.

A reported deadlock. The JVM detects monitor cycles and prints "Found one Java-level deadlock" with both threads and both locks, which makes this the easiest production problem to diagnose and the hardest to reach by accident.

The reason for taking several dumps: a thread appearing at the same stack frame in all three is genuinely stuck, while one that has moved was simply doing work when the sample was taken.

14. Heap dumps and memory leaks

Garbage collection behaviour is usually the first evidence of a leak; the heap dump is what identifies the cause. Heap usage naturally rises and falls in a sawtooth as garbage accumulates and is collected, so the raw graph proves nothing. What matters is the floor:

```
Healthy: post-collection occupancy returns to roughly the same level
Leaking: post-collection occupancy rises steadily over hours or days
         full collections become more frequent and reclaim less each time
         eventually: java.lang.OutOfMemoryError: Java heap space
```

A heap dump analysed in Eclipse MAT, the Memory Analyzer Tool, answers the question of what is holding the memory. The **dominator tree** is the view to use: it shows which objects would free the most if released, which is retained size rather than shallow size.

Causes that account for most leaks in practice:

- **An unbounded cache**, which is why the caching note treats an eviction policy as mandatory rather than optional.
- **A static collection** that only ever gets added to.
- **A `ThreadLocal` not cleaned up** on a pooled thread, so the value survives as long as the pool does.
- **A listener or callback never deregistered**, keeping its enclosing object alive.
- **A class loader leak** after repeated redeployment, which retains every class it loaded.

Worth knowing: **`OutOfMemoryError` is not always the heap**. The message says which limit was hit, and `unable to create new native thread`, `Metaspace` and `Direct buffer memory` are all different problems with different fixes. Reading the rest of the line before assuming a heap leak avoids hours spent looking in the wrong place.

15. Garbage collection

Enough to reason about production behaviour, which is what gets asked, rather than a tuning guide.

The **generational hypothesis** is the assumption everything rests on: most objects die young. So the heap is split, young collections are frequent and cheap because almost everything in the young generation is already garbage, and collections of the old generation are rarer and more expensive.

That split produces the collection kinds an interviewer will ask to have distinguished:

Collection	What it does
Young	Collects only the young generation. Frequent, short, and survivors are promoted toward the old generation
Mixed , in G1	Collects the young generation plus some old-generation regions chosen for how much garbage they hold
Full	Collects the whole heap, usually with a long stop-the-world pause. In G1 it is a fallback, and repeated full collections mean the concurrent work is not keeping up

The short answer to "young against full": young collections are the normal, cheap operation and their cost scales with how much survives, while a full collection touches everything and is the one that shows up as a latency spike.

Collector	Character
G1, the default since JDK 9	Region-based, aims at a configurable pause target. A reasonable default for most services
ZGC and Shenandoah	Concurrent, pause times largely independent of heap size, at the cost of more processor work
Parallel	Maximises throughput, accepts longer pauses
Serial	Small heaps and single-processor containers

What to look at in a GC log, in order: **pause durations**, **collection frequency**, and **whether old-generation occupancy after a full collection is trending upward**, which is section 14's leak signature.

The tuning position worth stating: set the heap size and a pause-time goal, then stop until there is evidence. Most problems that look like garbage collection problems are allocation problems or leaks, and no collector setting fixes an application that allocates too much. The container interaction from the cloud note applies here too: size the heap as a percentage of the container limit, leaving room for the non-heap memory the JVM also needs.

16. Profiling and flame graphs

Two kinds of profiler. An **instrumenting** profiler adds code around method calls, which gives exact counts and can slow the application substantially. A **sampling** profiler periodically captures stacks, so its overhead is small and roughly constant regardless of how much the application does. **Sampling profilers are the usual choice in production**, because that predictable overhead is what makes them safe to leave running; instrumenting profilers can still be used there when narrowly scoped or enabled only for a short window.

Tool	Character
JFR , Java Flight Recorder	Built into the JDK, overhead low enough to run continuously, records far more than stacks
async-profiler	Sampling profiler that avoids the safepoint bias affecting some JVM profilers, and can sample allocation, lock contention and wall-clock time as well as processor time

Safepoint bias is worth knowing as a term: profilers that can only sample at safepoints see a distorted picture, because safepoints are not evenly distributed through the code.

A **flame graph** visualises a set of stacks. The rules for reading one are short and often got wrong:

- **The x-axis is proportion of samples, not time.** The graph is not a timeline, and the left-to-right order carries no meaning beyond alphabetical grouping.
- **The y-axis is stack depth**, with callers below and callees above.
- **Width is the primary signal.** A wide frame means many samples were taken inside it or its children, so that is where the time went. Position in the stack still matters for working out which caller the samples arrived through.
- **Colour usually carries no meaning**, beyond making adjacent frames distinguishable.

Reading one: look for the widest plateaus. A wide frame with narrow children is doing the work itself and is where to optimise. A wide frame with one equally wide child is only calling through, so keep going up.

Continuous profiling keeps sampled, aggregated profiles from across the fleet under a retention policy, which means "this got slower after Tuesday's release" can be settled by comparing two flame graphs rather than by discussion.

17. Load testing

Four kinds, answering different questions:

Test	Question
Load	Does it meet its objectives at expected traffic
Stress	Where does it break, and how does it break
Soak	Does it survive hours of sustained traffic
Spike	What happens when traffic jumps suddenly

Stress testing earns particular attention, because knowing the breaking point is useful and knowing the **failure mode** at that point is what determines whether an overload becomes an outage. A system that sheds load and stays up at 150% of capacity is in a completely different position from one that queues until it exhausts memory.

The soak test finds what no short test can: memory leaks, file descriptor exhaustion, connection pools that slowly lose connections, and log volume filling a disk.

Four mistakes that make results misleading:

- **A load generator with coordinated omission**, from section 11, which is the most common one.
- **Not warming the JVM**. Code runs interpreted until the JIT compiles it, so the first minute of results is pessimistic and mixing it into the average hides real numbers.
- **Unrealistic data distribution**. Requesting the same key repeatedly gives a cache hit ratio no real workload will produce.
- **Extrapolating linearly from a scaled-down environment**. Contention and saturation are not linear, and the whole point of testing is to find where they stop being linear.

18. Chaos engineering

Deliberately injecting failure to verify the system behaves the way the design claims. It is **an experiment with a hypothesis**, not breaking things randomly, and describing it that way is what separates a considered answer from a fashionable one.

The method:

1. Define the steady state as a measurable metric, such as successful orders per minute.
2. Hypothesise that the metric will not change when a specific failure occurs.
3. Inject that failure.
4. Measure whether the hypothesis held.
5. Keep the scope of impact small, and have an abort condition ready.

The scope of impact has a standard name worth using, the **blast radius**: how much traffic, how many customers and which regions the experiment can affect if the hypothesis turns out to be wrong. Deciding it in advance, along with the condition that ends the experiment, is what makes this an engineering practice rather than an outage someone chose to cause.

What is worth injecting: terminating an instance, adding latency to a dependency, making a dependency return errors, exhausting a resource, partitioning the network, and skewing a clock.

The reason it is worth doing: **failure handling is code that never runs in normal operation**. Circuit breakers, timeouts, retries, fallbacks and degraded modes are all paths that are otherwise first executed during a real incident, by which point nobody knows whether they work. A retry that was misconfigured two years ago behaves identically to one that works, until the day it matters.

Start in a staging environment, then move to production with a small scope, because staging does not have production's traffic, data or dependencies, and those three are what usually produce the unexpected result.

19. Production readiness

A checklist worth having ready, because "how do you know it is ready for production" is a common question, and a prepared list answers it far better than an improvised one.

Observability

- Service level objectives defined, with the indicators actually implemented.
- A dashboard showing RED for the service and USE for the resources beneath it.
- Alerts on symptoms rather than causes, each with a runbook. For how objectives turn into alerts through error budgets and burn rate, see the Cloud and DevOps note rather than reworking it here.
- Structured logs carrying the trace identifier.
- Traces propagating across every hop, including asynchronous ones.

Operability

- Liveness and readiness probes that are distinct, with no dependency checks in liveness.
- Resource requests and limits set deliberately.
- A timeout on every outbound call.
- Graceful shutdown that finishes in-flight requests and stops accepting new ones.
- On-call rotation and escalation defined, with someone who has agreed to it.

Change safety

- Rollback tested rather than assumed.
- Database migrations following expand, migrate and contract.
- Feature flags for anything risky, with a plan to remove them.

Capacity and data

- A load test result showing headroom above expected peak.
- Backups verified by performing a restore, since an unverified backup is an assumption rather than a fact.
- Data retention defined, and personal data handling agreed.

20. Interview questions

Each question below has two parts. The **opening** is what to say first, and it is kept to roughly thirty to forty seconds, because that is about as long as an interviewer listens before wanting to steer. Its last sentence points at the deeper material rather than closing the topic, so the natural follow-up lands on ground we have already prepared. The **follow-up** is what to say when they take it.

Saying the whole thing at once is the common mistake. It reads as a prepared speech, and it spends material that would have earned a second question.

"What is the difference between monitoring and observability?"

Monitoring watches signals we chose in advance for failures we anticipated in advance, so it tells us whether a known bad condition is happening. Observability is having enough telemetry and context to investigate behaviour nobody predicted. A rising error rate alert tells me something is wrong; traces, structured logs and dependency metrics are what let me work out why. The test I actually use is whether a new question can be answered without shipping new code.

If they go deeper:

If every investigation starts by adding a metric and waiting for a deploy, we have monitoring. The underlying framing is known unknowns against unknown unknowns: we know a database can get slow, so we watch it, and observability is for the combination of conditions nobody thought about. I would also be honest that the two are not a clean boundary, and that collecting three kinds of telemetry does not confer observability on its own. What does is enough detail, with enough context attached, to slice the data along a dimension nobody anticipated.

"Why can you not average percentiles?"

Because a percentile is a position in a distribution rather than a quantity, so the mean of ten p99 values is not the p99 of the combined traffic, and it is usually optimistic. It matters because the tail is what users feel, so a dashboard averaging p99s across pods reports a comfortable number while people are experiencing something else. Whether we can avoid it comes down to how the metric is exported.

If they go deeper:

A summary computes quantiles inside each process, which makes the number exact for that instance and impossible to combine across instances. A histogram exports counts per bucket instead, and buckets add together cleanly, so the percentile is estimated at query time from the whole distribution. I would call that an estimate deliberately, because its accuracy depends on where the bucket boundaries fall, but it does describe the service rather than one instance. In Micrometer it is the difference between publishing a percentile histogram and computing percentiles client-side, and for service-level latency across several instances the histogram is usually the better choice.

"What is cardinality and why does it matter?"

Cardinality is the number of distinct time series, which is the product of every combination of label values. One unbounded label is the most common way a metrics system gets destroyed: a request counter labelled by method, status and endpoint is a few thousand series and fine, and adding user identifier multiplies that by the user count and produces billions. The backend then stops working at the moment it is most needed. So the rule I follow is that labels have to be bounded and small.

If they go deeper:

Safe labels are method, status class, region and the endpoint template rather than the resolved path, because slash orders slash twelve thousand as a label value creates one series per order. Anything high cardinality generally belongs in logs or traces, which are built for it, though with their own cost controls and the same rules about personal data. That is really the division of labour between the three signals: metrics answer aggregate questions cheaply at low cardinality, and logs and traces answer specific questions expensively at high cardinality.

"A production service is slow. Walk me through what you do."

I start at the boundary metrics, because the first question is whether it is us or a dependency. That means RED for our own endpoints and the same for every outbound call, since a dependency's latency rising while ours follows it settles the question immediately. Then a trace of a slow request, to see which hop actually consumed the time rather than reasoning about it. If it turns out to be inside our process, the tooling changes.

If they go deeper:

Inside the process the next step is a thread dump, and I take three about five seconds apart so a thread that is genuinely stuck can be told apart from one that happened to be busy. Many threads runnable inside a socket read means they are blocked in native input and output waiting on something remote rather than using the processor, which is usually a missing timeout, and many blocked on the same monitor means contention with the holder's stack showing where. If it is processor-bound rather than waiting, then a profile, and I would use JFR or async-profiler because they sample and are safe to run in production. Reading the flame graph, width is the primary signal, since the x-axis is proportion of samples rather than elapsed time.

"What is coordinated omission?"

A measurement error that makes load test results look much better than reality. A typical load generator sends a request, waits for the response, then sends the next, so when the system stalls for a second the generator stalls with it and never sends the hundred requests it should have. Those requests never appear in the results, and neither does the queuing a real user would have experienced, so the recorded distribution omits precisely the worst period. That is where the name comes from, and it has a fix and a matching effect on the service side.

If they go deeper:

The correction is to measure latency against the intended send schedule rather than the actual one, so a request that could not be sent still accrues the wait it should have. wrk2 and HdrHistogram both do this. The service-side counterpart is tail latency amplification: a request fanning out to ten dependencies, each with a one percent chance of being slow, has about a nine and a half percent chance that at least one is, assuming those events are independent. I would put that as an approximation rather than a rule: what is a one-in-a-hundred event for one dependency becomes close to a one-in-ten event for the request as a whole. In practice dependencies are often correlated through a shared database or network path, which makes the real figure worse rather than better.

"How do you find a memory leak in a Java service?"

Garbage collection behaviour is usually the first evidence, and the heap dump is what identifies the cause. The raw heap graph proves nothing on its own, because usage naturally sawtooths as garbage accumulates and is collected. What matters is the floor: if occupancy after a full collection returns to roughly the same level it is healthy, and if that floor rises steadily over hours while collections get more frequent and reclaim less each time, that is a leak heading for an OutOfMemoryError. At that point I want a heap dump.

If they go deeper:

I would want it captured automatically with HeapDumpOnOutOfMemoryError and a path, rather than hoping someone is at a keyboard when it happens, and then the dominator tree in Eclipse MAT, the Memory Analyzer Tool, to see which objects retain the most. The causes I look for first are an unbounded cache, a static collection only ever added to, a ThreadLocal never cleaned up on a pooled thread, and a listener never deregistered. One thing I check before assuming any of that is the actual OutOfMemoryError message, because unable to create new native thread, metaspace and direct buffer memory are different problems with different fixes.

"What would you capture before restarting a failing service?"

Everything that exists only inside the running process, because that is exactly what the restart destroys and the pressure in an incident is always to restart first. The telemetry already sent survives, so we are not blind afterwards, but the in-process state is gone. For a JVM that means three thread dumps a few seconds apart, the heap information, and a JFR recording, plus a heap dump if memory is the suspicion. Two of those want configuring in advance.

If they go deeper:

The heap dump is the one to think about rather than reach for, because it pauses the process and produces a file the size of the live heap. The two I would have configured already are `HeapDumpOnOutOfMemoryError` with a path, and garbage collection logging to a rotating file, because both capture evidence when nobody is watching. In Kubernetes this gets harder than it sounds, since a failing liveness probe restarts the container and takes its filesystem with it, so the dumps need a volume that survives the restart, a `preStop` hook, or a termination grace period long enough to collect anything at all.

"What is chaos engineering, and is it just breaking things?"

No, it is an experiment with a hypothesis. We define steady state as a measurable metric such as successful orders per minute, state that it will not change when a specific failure occurs, inject that failure, and measure whether the hypothesis held, with the blast radius and an abort condition both decided in advance. What justifies doing it at all is that failure handling is code that never runs in normal operation.

If they go deeper:

Circuit breakers, timeouts, retries and fallbacks are all paths first exercised during a real incident, and a retry misconfigured two years ago looks identical to one that works until the day it matters. Worth injecting: terminating an instance, adding latency to a dependency, making one return errors, exhausting a resource, partitioning the network. I would start in staging and then move to production with a small blast radius, because staging does not have production's traffic, data or dependencies, and those three are what usually produce the unexpected result.

21. Common mistakes

- Alerting on an average latency, which cannot move when the tail doubles.
- Averaging percentiles across instances, which reports a comfortable number that no user experiences.
- Exporting client-side percentiles from a service with more than one instance, when a histogram would aggregate correctly.
- Putting an unbounded value such as a user or request identifier into a metric label, which multiplies cardinality until the metrics backend fails.
- Using the resolved path rather than the endpoint template as a label, which creates one time series per resource.
- Logging a line per request in order to count requests, which is the most expensive counter available.
- Emitting twenty INFO lines per request, so INFO becomes DEBUG and the log stops being readable.
- Writing log messages as sentences with the variables inside them, so every query needs a regular expression.
- Losing trace context across an executor or a queue, then concluding tracing is broken when it is an unwrapped thread pool.

- Restarting a failing process before capturing anything, which destroys every piece of evidence that existed only inside it.
- Taking one thread dump, which cannot distinguish a stuck thread from a busy one.
- Reading a flame graph left to right as a timeline, when the x-axis is proportion of samples.
- Putting a customer identifier or a raw query into a span because traces tolerate high cardinality, when the privacy rules that govern logs govern traces too.
- Assuming every `OutOfMemoryError` is a heap leak, when metaspace, native threads and direct buffers produce the same class with different causes.
- Tuning garbage collection settings before establishing whether the problem is allocation rate or a leak.
- Load testing with a generator that suffers coordinated omission, which hides exactly the period being tested for.
- Load testing without warming the JVM, so interpreted execution is averaged into the result.
- Treating a backup as verified without ever having performed a restore.

22. Quick reference

Term	Meaning
Monitoring against observability	Known failure modes, against answering unanticipated questions
Counter	Only increases. Query the rate, never the value
Gauge	A point-in-time value that rises and falls
Histogram	Bucket counts, which aggregate across instances. The percentile is an estimate set by the bucket boundaries
Summary	Quantiles computed in-process, which cannot be aggregated
Cardinality	Distinct time series, the product of all label combinations
Endpoint template	<code>/orders/{id}</code> as a label, never <code>/orders/12345</code>
RED	Rate, errors, duration. For request-driven services
USE	Utilisation, saturation, errors. For resources
Four golden signals	Latency, traffic, errors, saturation
Structured logging	Key and value fields with a constant message, so logs are queryable
MDC	Context attached to every log line in a request, such as the trace identifier
OTLP	The OpenTelemetry Protocol. Carries all three signals between components, over gRPC or HTTP
Collector	Separate process that receives, filters, redacts, tail samples and exports. Optional, and what keeps backend details out of every service

Term	Meaning
Head against tail sampling	Decide at the start, which the SDK can do, against deciding once the trace is complete, which needs the collector. Cheap against keeping the rare failures
p99	A common latency objective for user-facing services. Never averaged across instances
Tail amplification	Fanning out to <code>n</code> dependencies multiplies exposure to each one's tail. The usual figure assumes independence, and correlation makes it worse
Coordinated omission	A load generator stalling with the system, so the worst period goes unrecorded
<code>jcmd Thread.print</code>	A thread dump. Take three, seconds apart
<code>BLOCKED</code> against <code>RUNNABLE</code> in <code>socketRead</code>	Lock contention, against waiting on a remote call. <code>RUNNABLE</code> does not imply processor use
Dominator tree	The Eclipse Memory Analyzer Tool view showing what retains the most memory
Rising post-GC floor	The first evidence of a memory leak. The heap dump identifies the cause
JFR	Java Flight Recorder. Low enough overhead to run continuously
Safepoint bias	Distortion in profilers that can only sample at safepoints
Flame graph width	Proportion of samples. The primary signal, and not a measure of elapsed time
Soak test	Sustained load, which is what finds leaks and resource exhaustion
Chaos experiment	Steady-state metric, hypothesis, injection, measurement, small scope
Blast radius	How much traffic and how many customers an experiment can affect. Decided before it runs
Resource attributes	Service, version, instance, environment, region. Set once per process, on every signal
Young against full collection	Young generation only and cheap, against the whole heap and the source of the pause

Symptom	Where to look
Latency up, no error rate change	Dependency metrics first, then a trace of a slow request
Dashboard healthy, users complaining	Averaged percentiles, or an average instead of a distribution
Metrics backend slow or falling over	Cardinality, and a recently added label
Trace stops halfway through the system	Context dropped at that hop, usually an async boundary
Threads climbing, throughput falling	Thread dump. Expect <code>socketRead</code> and a missing timeout
Heap rising over days	GC log floor, then a heap dump and the dominator tree
<code>OutOfMemoryError</code> with a healthy heap	Read the message: metaspace, native threads or direct buffers

Symptom	Where to look
Fast in a benchmark, slow in production	Coordinated omission, or a JVM never warmed
Regression after a release	Compare flame graphs from before and after
Recovery slower than expected	The failure path has never been exercised. Inject it deliberately